COLLEGE DE PARIS SUPERIEUR

Note : /20

CdPS	Contrôle Continu	Durée : 2h Credit: 2
Année Scolaire : 2024-2025	Épreuve : ARCHITECTURE ET PROGRAMMATION EN ASSEMBLEUR	Classe : B2 - A (IT)

Instructions Générales

- Lisez attentivement chaque question avant de répondre.
- Respectez les consignes spécifiques données pour chaque exercice.
- Les programmes doivent être écrits en syntaxe NASM pour l'architecture x86_64.
- La rigueur syntaxique, la clarté, la lisibilité du code et les commentaires sont pris en compte dans la notation.

Partie I : Questions de compréhension (10 pts)

1. (1 pt) Définissez un microcontrôleur et précisez son rôle dans les systèmes embarqués.
2. (1 pt) Présentez brièvement l'architecture générale d'un microcontrôleur AVR.
3. (1 pt) Quelles sont les principales mémoires présentes dans un microcontrôleur AVR ?
4. (1 pt) Décrivez brièvement le rôle de l'ALU dans un microcontrôleur.
5. (1 pt) Expliquez le concept de compilation croisée dans le développement pour AVR.
6. (1 pt) Qu'est-ce qu'un registre ? Donnez deux exemples de registres 64 bits standards.
7. (1 pt) Quelle est la différence entre les registres `RAX` et `EAX` ? Dans quel cas utiliser l'un plutôt que l'autre ?
8. (1 pt) Quelle est la différence entre les instructions `mov` et `lea` ? Donnez un exemple d'utilisation de `lea`.
9. (1 pt) Donnez deux outils permettant de programmer et déployer un code sur un microcontrôleur AVR.
10. (1 pt) Comparez la structure d'un programme assembleur x64 à celle d'un programme assembleur AVR.

Partie II : Exercices pratiques (10 pts)

Complétez le programme NASM en assembleur x64 selon les instructions données dans les différents exercices. Utilisez le format suivant pour chaque exercice :

```
section .data
    ; Déclaration des variables (si nécessaire)
    ; Par exemple : var1 db 10  ; Déclaration d'une variable de type byte avec la valeur 10

section .text
global main
main:
    ; Instructions à compléter
    ; Par exemple : mov rax, [var1]   ; Charger la valeur de la variable dans un registre
    ; Vous pouvez ajouter des instructions comme l'addition, la soustraction,
    ; ou des déplacements de valeurs.

    ret
```


Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

Exercice 1 : Écriture d'un programme simple (2 pts)

Écrivez un programme NASM en assembleur x64 qui :

- place la valeur 5 dans le registre `RAX`,
- place la valeur 3 dans le registre `RBX`,
- additionne les deux registres et stocke le résultat dans `RCX`.

Exercice 2 : Correction de code erroné (2 pts)

Le code suivant contient des erreurs. Corrigez-le pour qu'il fonctionne sous NASM x64.

```
section .text
global main
main:
    mov rax, 5
    mov rbx 3
    ADD RAX RBX
    mov rcx, rax
    ret
```

Exercice 3 : Compilation (2 pts)

- Expliquez comment compiler un programme NASM sous Windows en utilisant `nasm` et `gcc (MinGW-w64)`.
- Mentionnez au moins deux commandes à exécuter dans le terminal.

Exercice 4 : Adressage mémoire (2 pts)

Écrivez un programme NASM x64 qui :

- déclare une variable `val` contenant l'entier 42,
- charge cette valeur en mémoire dans le registre `RAX`,
- incrémente `RAX` de 1.

Exercice 5 : Lecture et écriture mémoire (2 pts)

Complétez le programme suivant pour :

- copier la valeur 100 dans `RAX`,
- stocker cette valeur dans une variable appelée `memoire`,
- remettre la valeur de `memoire` dans `RBX`.

"Le secret du succès est de faire les choses communes exceptionnellement bien." John D. Rockefeller

Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

CdPS	Corrigé-type du Contrôle Continu	Durée : 2h Credit: 2
Année Scolaire : 2024-2025	Épreuve : ARCHITECTURE ET PROGRAMMATION EN ASSEMBLEUR	Classe : B2 - A (IT)

Partie I : Questions de compréhension (10 pts)

1. **(1 pt) Qu'est-ce qu'un microcontrôleur ?**

   Un **microcontrôleur** est un circuit intégré programmable, regroupant dans une seule puce :

   - une unité centrale de traitement (**CPU**) : exécute les instructions ;
   - une mémoire **Flash** (programme), **SRAM** (données volatiles), et parfois **EEPROM** (données non volatiles) ;
   - des interfaces d'entrée/sortie numériques et analogiques (GPIO, ADC, PWM, UART, SPI, I2C, etc.) ;
   - un contrôleur d'interruptions et des timers.

   Il est utilisé dans les systèmes embarqués pour le contrôle d'appareils physiques (robots, capteurs, domotique...).

2. **(1 pt) Architecture interne d'un microcontrôleur AVR**

   Les microcontrôleurs AVR (comme l'ATmega328P) reposent sur une architecture Harvard modifiée :

   - 32 registres de travail (R0 à R31) accessibles directement par l'ALU ;
   - une mémoire Flash (jusqu'à 256 Ko), une SRAM (jusqu'à 8 Ko), une EEPROM ;
   - une unité de traitement : ALU, compteur ordinal (PC), registre d'état (SREG) ;
   - un ensemble de périphériques intégrés (UART, SPI, I2C, timers, convertisseur ADC) ;
   - gestion d'interruptions via vecteurs et priorités.

3. **(1 pt) Différents types de mémoire dans un AVR**

   - **Mémoire Flash** : non-volatile, stocke le programme ; accessible uniquement en lecture lors de l'exécution ;
   - **SRAM** : mémoire volatile pour stocker les variables à l'exécution ;
   - **EEPROM** : non-volatile, utilisée pour les données à conserver entre deux sessions (paramètres utilisateurs, calibrations, etc.) ;
   - **Registres** : mémoire très rapide interne au CPU (R0–R31).

4. **(1 pt) Rôle de l'ALU dans un microcontrôleur**

   L'**unité arithmétique et logique** (ALU) est le cœur de calcul du processeur. Elle exécute :

   - les opérations **arithmétiques** : addition, soustraction, multiplication (et parfois division) ;
   - les opérations **logiques** : ET, OU, NON, XOR ;
   - les **comparaisons** et tests de bits ;
   - les opérations de **décalage** (shift) ou **rotation**.

5. **(1 pt) Compilation croisée : définition et justification**

   La **compilation croisée** consiste à compiler un programme destiné à une plateforme matérielle différente de celle du compilateur.

   - Exemple : utiliser `avr-gcc` sur un PC x64 pour produire un binaire destiné à un microcontrôleur AVR ;
   - Justification : le microcontrôleur n'a pas la puissance ni les outils pour se compiler lui-même (contrairement aux ordinateurs classiques) ;
   - Avantage : permet de développer sur des machines puissantes pour des cibles légères.

6. **(1 pt) Qu'est-ce qu'un registre ? Donner deux exemples.**

   Un **registre** est une petite zone mémoire ultra-rapide interne au processeur.

   - En assembleur x64 : `RAX`, `RCX`, `RSP`, etc. ;
   - En assembleur AVR : `R16`, `R17`, etc. ;
   - Ils stockent des opérandes, résultats d'opérations, adresses, compteurs, etc.

Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

7. **(1 pt) Différence entre RAX et EAX**
   - `RAX` est un registre de 64 bits ;
   - `EAX` représente les 32 bits de poids faible de `RAX` ;
   - De même, `AX` (16 bits), `AL` (8 bits inférieurs), `AH` (8 bits supérieurs) ;
   - Cela permet d'accéder à différentes parties du même registre.

8. **(1 pt) Différence entre `mov` et `lea`**

```
mov rax, [rbx]      ; charge dans RAX la valeur pointée par l'adresse RBX
lea rax, [rbx+4]    ; charge dans RAX l'adresse calculée (RBX+4), sans accès mémoire
```

   `lea` signifie "Load Effective Address" : utile pour l'arithmétique d'adresses.

9. **(1 pt) Citer 2 outils utiles pour programmer un microcontrôleur**
   - `avr-gcc` : compilateur pour le langage C adapté aux AVR ;
   - `AVRDUDE` : téléversement du binaire vers la carte (via USB) ;
   - `Atmel Studio`, `PlatformIO`, ou `Arduino IDE` : environnement de développement ;
   - `SimAVR` ou `AVR Simulator IDE` : simulateurs pour tester sans matériel.

10. **(1 pt) Deux différences entre programme assembleur x64 et programme AVR**
    - **x64** :
      - architecture CISC, 64 bits, instructions complexes ;
      - adressage mémoire étendu, pile logicielle, conventions d'appel ;
      - présence d'un OS (Windows/Linux).
    - **AVR** :
      - architecture RISC, instructions simples et rapides ;
      - accès direct aux registres, usage matériel embarqué ;
      - pas de système d'exploitation, exécution en temps réel.

Partie II : Exercices pratiques (10 pts)

Exercice 1 : Addition de deux constantes

```
section .text
global main
main:
    mov rax, 5        ; charger 5 dans le registre RAX
    mov rbx, 3        ; charger 3 dans le registre RBX
    add rax, rbx      ; additionner les deux valeurs, résultat dans RAX
    ret               ; retourner (fin de programme)
```

Exercice 2 : Corriger le code erroné

```
section .text
global main
main:
    mov rax, 5        ; charger 5
    mov rbx, 3        ; charger 3
    add rax, rbx      ; rax = rax + rbx = 8
    mov rcx, rax      ; rcx reçoit 8
    ret
```


Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

Exercice 3 : Compilation sous Windows

```
# Étape 1 : assemblage
nasm -f win64 programme.asm -o programme.obj

# Étape 2 : édition de lien
gcc programme.obj -o programme.exe
```

Exercice 4 : Lecture mémoire

```
section .data
    val dq 42        ; déclaration d'un entier 64 bits

section .text
global main
main:
    mov rax, [val]   ; charger la valeur 42 depuis la mémoire
    inc rax          ; incrémenter rax → 43
    ret
```

Exercice 5 : Lecture et écriture mémoire

```
section .data
    memoire dq 0     ; mémoire initialisée à 0

section .text
global main
main:
    mov rax, 100          ; charger 100 dans rax
    mov [memoire], rax    ; écrire 100 dans la mémoire
    mov rbx, [memoire]    ; relire 100 dans rbx
    ret
```


Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net